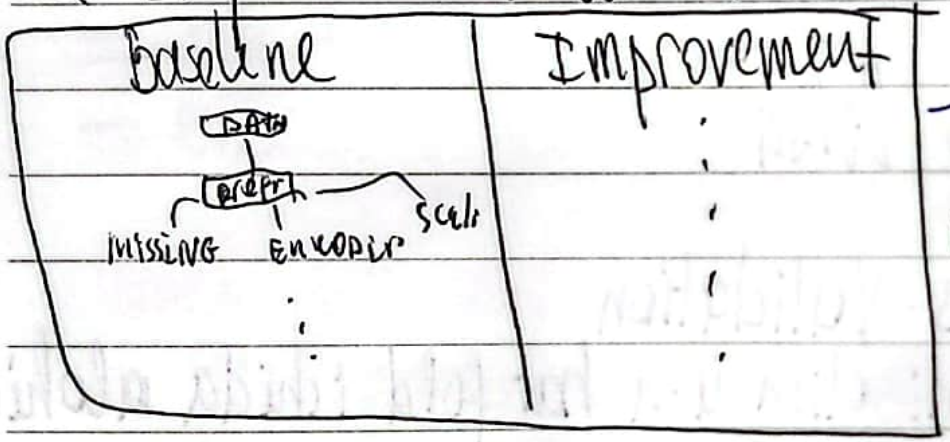
3 - dars : Model Improvement

Flowchart chizish Report uchun. 1 ta Baseline uchun
2 - Improvement uchun Ram



Advanced Missing Value Treatment — Bu tushib qolgan qiymatlarni ilg'i bolicha haqiqatga yaqinlashtirish uchun organsayapmiz

- Lesson 1 Decision making metrics (precision & recall)
- Lesson 2 Advanced Missing Values === Ishonarli Dataset

No.

Advanced Feature Engineering

Future Engineering nima? -> Bu Machine learning modelining ishlashini yaxshilash uchun mavjud ma'lumotlarni o'zgartirish, tozalash, yoki yangi foydali featurelar yaratish jarayonidir. Bu jarayonda missing value handling encoding, scaling va yangi feature yaratish kabi usullar qo'llaniladi. Yaxshi feature engineering model aniqligini sezilarli darajada oshirishi mumkin.

- Asosiy maqsad
  ↳ Datani uning o'zniga undan xam ma'nolisi bilan almashtirish. Yoki yoniga (eskisi ham qolsin) qo'shimcha uni ham ishlatayilik.

- Asosiy tur / bo'limlari:
  I. High-Order Features - Interactions & Polynomial Features
  II. Ratios, Log Transforms & Domain-Specific Features
  III. Target Encoding with Leakage Prevention
  IV. Frequency / Count Encoding
  V. Time-Based Rolling Features & Lag Features

Advanced Feature Engineering turlari va amaliy:

- High-Order Feature Interactions & Polynomial Features
  Bunda yangi featureler yaratiladi. Ya'ni featurelarni bir-biri bilan munosabatiga ko'ra yuqori darajadagi featurelarni yaratish mumkin bo'ladi.

No.

= = = # # # = = =

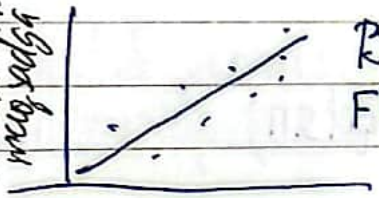
Ichki resurslardan foydalangan holda matematik amallar yoki formulalar orqali yangi feature lar yaratish.

Masalan:

- Yillik oylik = Oylik * 12
- Polynomial = x^2, x^4

```
from sklearn.preprocessing import
PolynomialFeatures
poly = PolynomialFeatures(degree=2,
include_bias=False)
X_poly = poly.fit_transform(X)
print(poly.get_feature_names_out())
```

Asosiy foydali === Linear Regression (Linear o'lasi uchun ancha foydali bo'ladi)




```
feature_names = poly.get_
feature_names_out(X.columns)
X_poly_df = pd.DataFrame(X_poly,
columns = feature_names)
df_poly = pd.concat([X_poly_df, y],
axis=1)
df_poly.info()
```

Bu yerdagi (x) ni biz df CC'agi, sutun yaratish nuqtaqal dan bersak ma'lum bir ma'noga ega bo'ladi va hammaga ma'lum bo'lmagan ma'noga ega bo'ladi

- Ratios, Log Transforms & Domain-Specific Features
  ↳ Numerical feature larni, pattern larni o'rganish uchun biz turdan boshqa turga o'tkazish

Masalan: Log transformation; Domain (xaftalik oylik muhimroq → yillik / 52)

Asosiy foydali === Barcha algoritmlarda (Ratio)

Analogy: Oylik 2000 $

↳ dekin bu yetarli emas bu maoshni yaxshi yoki yomon deyishga (xaftalik, yillik) ---> ratio! - Biror narsani Biror narsadagi ulushini ko'rsatish

Ratios, Log Transforms & Domain Specific Features

```
import numpy as np
df['goals_per_90'] = df['goals'] / df['minutes'] * 90
df['xG_per_shot'] = df['xG'] / df['shots']
df['log_salary'] = np.log1p(df['salary'])
df['goals_per_game'] = df['goals'] / df['game']
```

— Target Encoding with Leakage Prevention

↳ Categorical qiymatlar uchun → mean of target
(Yani Encoding bolayotgan ustunni target qiymatini mos elementlarini o'rta arifmetigi bilan to'ldiradi)

Masalan:

Marka	Target
BMW	10,000
BMW	20,000
Tesla	30,000
Tesla	40,000

$$\frac{10,000 + 20,000}{2} = 15,000$$

$$\frac{30,000 + 40,000}{2} = 35,000$$

Marka (Encoded)	Narxi
15,000	10,000
15,000	20,000
35,000	30,000
35,000	40,000

training zor ↗
testing past ↘

→ bunda a sida ham zor chiqadi, lekin u keyin bilinadi (deployment yoki hatto overfittingga ham o'xshashi mumkin)

- Overfitting vs Leakage
- Overfitting === data notog'ri ma'lumot/ni ko'p o'rganган.
- Yodlab olgan o'quvchi
- Leakage === to'g'ri lekin trainingda bo'lmasligi kerak bo'lgan ma'lumotlarni ko'p o'rganган.

[sizib chiqish]

- Imtixon savollarini imtixondan oldin qo'lga chiqib ketishi.

Leakage ga olib boruvchi eng ko'p takrorlanadigan yo'l bu → Target qiymatni ham Feature Engineringda ishlatib qo'ygan bo'lasizlar.

↳ Demak FE yaratayot ekanmizda Target qiymatni ishlatmasligimiz kerak.

- Target qiymatdan feature yaratishda foydalanmang
  - Leakage bo'ladi
  - Classlardagi xatolik bo'ladi. → Masalan bir classda target qiymatimiz orqali biron feature yasagan bo'lsak va keyinchalik df.drop('target', axis=1) deganimizda u targetni qidiradi va u yo'q deya ziddiyat yuzaga kelad-di.

- Frequency / Count Encoding

Bu kategorik qiymatlarni ular jadvalda necha marta takrorlanganiga qarab raqamga aylantirish usulidir.
- Categorical features uchun ishlaydi

No.

Mijoz	Shahar
1	Toshkent
2	Toshkent
3	Toshkent
4	Samarqand
5	Samarqand
6	Buxoro

Frequency Encoding qanday ishlaydi?

1. Toshkent : 3 marta kelgan
2. Samarqand : 2 marta kelgan
3. Buxoro : 1 marta kelgan

Natija (kodlangan jadval):

Shahar (Frequency Encoded)	Izoh
3	Toshkent 3 marta
3	takrorlangan
3	
2	Samarqand 2 marta
2	
1	Buxoro 1 marta

(Bazan bu sonlar umumiy qatorlar soniga bolib yuboriladi, yani Toshkent = 3/6 = 0,5 kelinishda foizda hisoblanadi.)

No.

Target Encoding with leakage Prevention

```
from category_encoders import TargetEncoder
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("target_enc", TargetEncoder(cols=['position'])),
    ("model", RandomForestRegressor())
])
scores = cross_val_score(pipeline, X, y, cv=5)
print(scores)
```

Bu yerda

Har bir train/test bölinishida: encoder faqat train data bilan o'rganadi, testga faqat qo'llanadi

yani test ma'lumotidan foyda olinmaydi. leakage yo'q

Bu datasetni 5 qismga bo'ladi.

Misol: Fold 1: train → test
Fold 2: train → test
⋮

Har safar:
1. Encoder trainda o'rganadi
2. Test encode qilinadi
3. Model baholanadi

5 oxirida 5 ta score chiqadi.

Frequency Encoding

```
freq = df['club'].value_counts(normalize=True)
df['club_freq'] = df['club'].map(freq)
```

sonni emas, foizni hisobla

map(freq) qiymatlarni shu foiz bilan almashtiradi.

- Time-Based Rolling Features & Lag Features
- Time-series yoki performancedan foydalangan holatda featurelar yaratish.

↳ Konkret 2000-yil yoki 2001-yil degan narsa bo'lmaydi. Bu umumiy holatda olinadi. Vaqt davom etadi pastda davomiylik bo'yicha hisoblanadi.

- Lag 1 = previous game's xG
- Lag 3 = average of last 3 games

Example: 1) last 5 matches, 2) performance trends
3) consistency (o'zgaruvchilik misol: Codewarsdan misol ish-tash o'zgaruvchi)

No.

har bir qiymatni 1 qadam pastga suradi!

O'tmishdagi tendensiyalar ⟷ O'tmishdagi nuqtalar

Time-Based Rolling & Lagging

(vaqtni tizimlash (avval to'g'ri kotma bitik o'rnatamiz)

```
df = df.sort_values(['player_id', 'match_date'])
```

har bir player'ni alohida-alohida timeline sifatida ko'ramiz

har bir qiymatni 1 qadam pastga suradi

```
df['xG_lag1'] = df.groupby('player_id')['xG'].shift(1)
df['xG_rolling3'] = df.groupby('player_id')['xG'].
                    rolling(3).mean().reset_index(0, drop=True)
```

qaysi ustun bo'l ishlayapmiz

Bu Multi Index qaytaradi

natija: player_id (A, A, B) original_index (0, 1, 2) NaN, avg, NaN

By yerda ota index bor: player_id, eski index

player_id indexini ~~olib tashla~~ taula

player_id'ni butunlay olib tashla

= = = = = = # # # = = = = = = =

real analogiya: Sen Excelda ota header qilib yubording ☺ bittasini olib tashlash kerak! reset_index(0, drop=True) ortiqcha header'ni o'chir

```
X = df.drop('Churn', axis=1)
y = df['Churn']
```

X → barcha ustunlar (features), churndan tashqari

y → target (yani bashorat qilinadigan qiymat - churn)

X = df[['age']] → bu ota qoovsdan iborat ustun degidir. Yani model "age" orqali o'rganadi

PolynomialFeatures nima qiladi?

```
from sklearn.preprocessing import PolynomialFeatures
```

Bu tool yangi feature'lar yaratadi.

Masalan: agar age = x bolsa, u quyidagilarni qo'shadi: x (original), x^2 (kvadrat)

No.

Polynomial model yaratish

▲ poly = PolynomialFeatures(degree=2, include_bias=False)

Bu yerda:
- degree = 2 -> maksimal daraja 2 (yani x^2 gacha)
- include_bias = False -> (constant column) qoshmaydi
  Agar "True" bolsa 1 ni ham qoshadi

Masalan: Agar X = [[5]] bolsa => [5, 25]
include_bias = True => [1, 5, 25]

Linear: $y = a*x + b$, Polynomial: $y = a*x^2 + b*x + c$
Bu yerda c => constant (Intercept)

▲ feature_names = poly.get_feature_names_out(X.columns)

X_poly_df = pd.DataFrame(X_poly, columns = feature_names)

df_poly = pd.concat([X_poly_df, y], axis=1)
df_poly.info()

Polynomial qilingandan keyin yangi featurelar paydo boladi: age <-> age^2 => ['age', 'age^2']

Agar bu qadamni qilmasang: DataFrame ustunli: 0, 1, 2 bolib qoladi, tushunish qiyin boladi.

No.

Numpy → DataFrame: juda muhim qadam
X_poly → oddiy array (numpy),
uni DataFramega aylantirsak. Natija:

age:	age^2
25	625
30	900

Endi ustun nomlari bor, ishlash oson.

```
df_poly = pd.concat([X_poly_df, y], axis=1)
```

X_poly_df → featurlar, y → target (Churn)

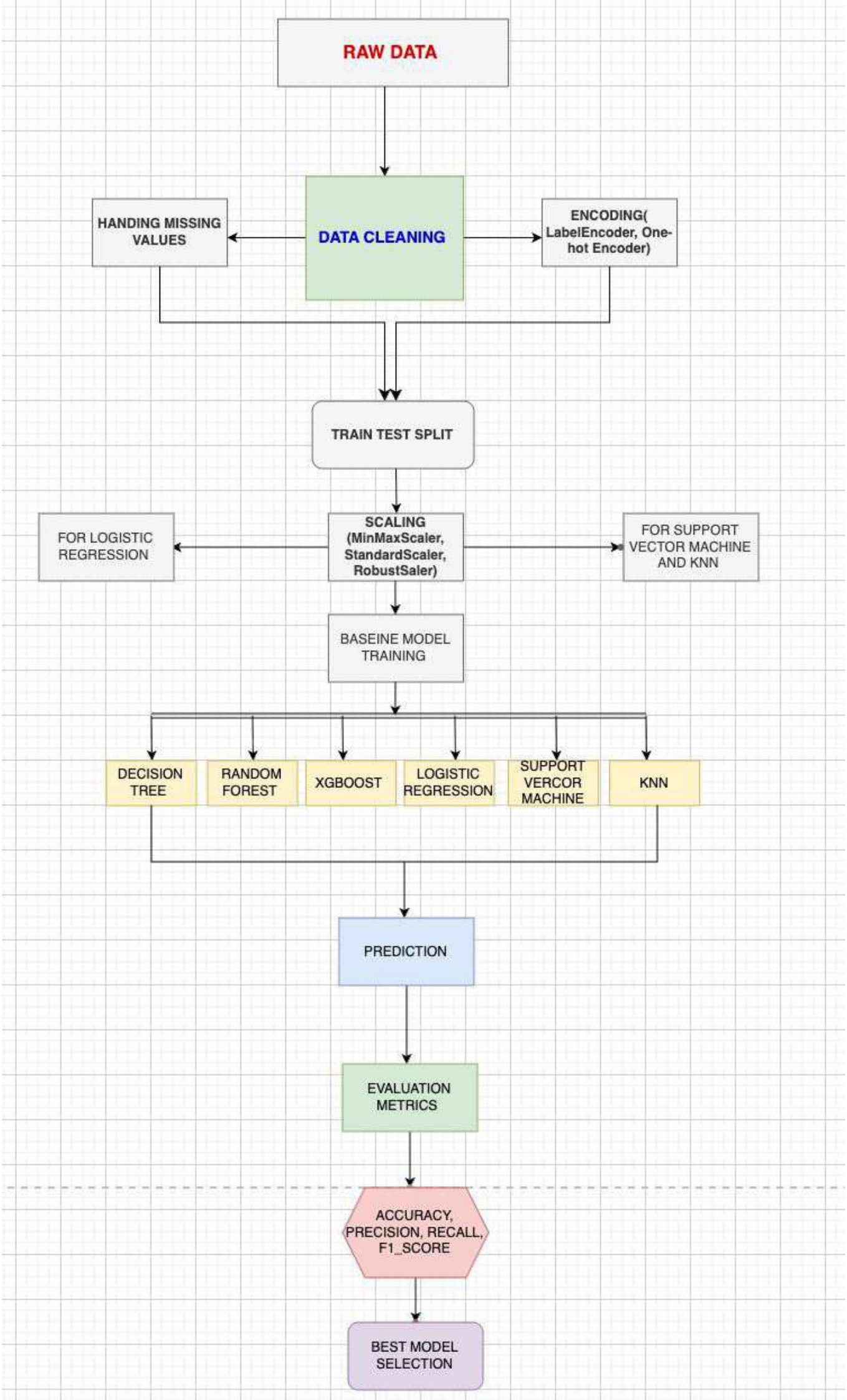
Ularni yonma-yon qo'shadi

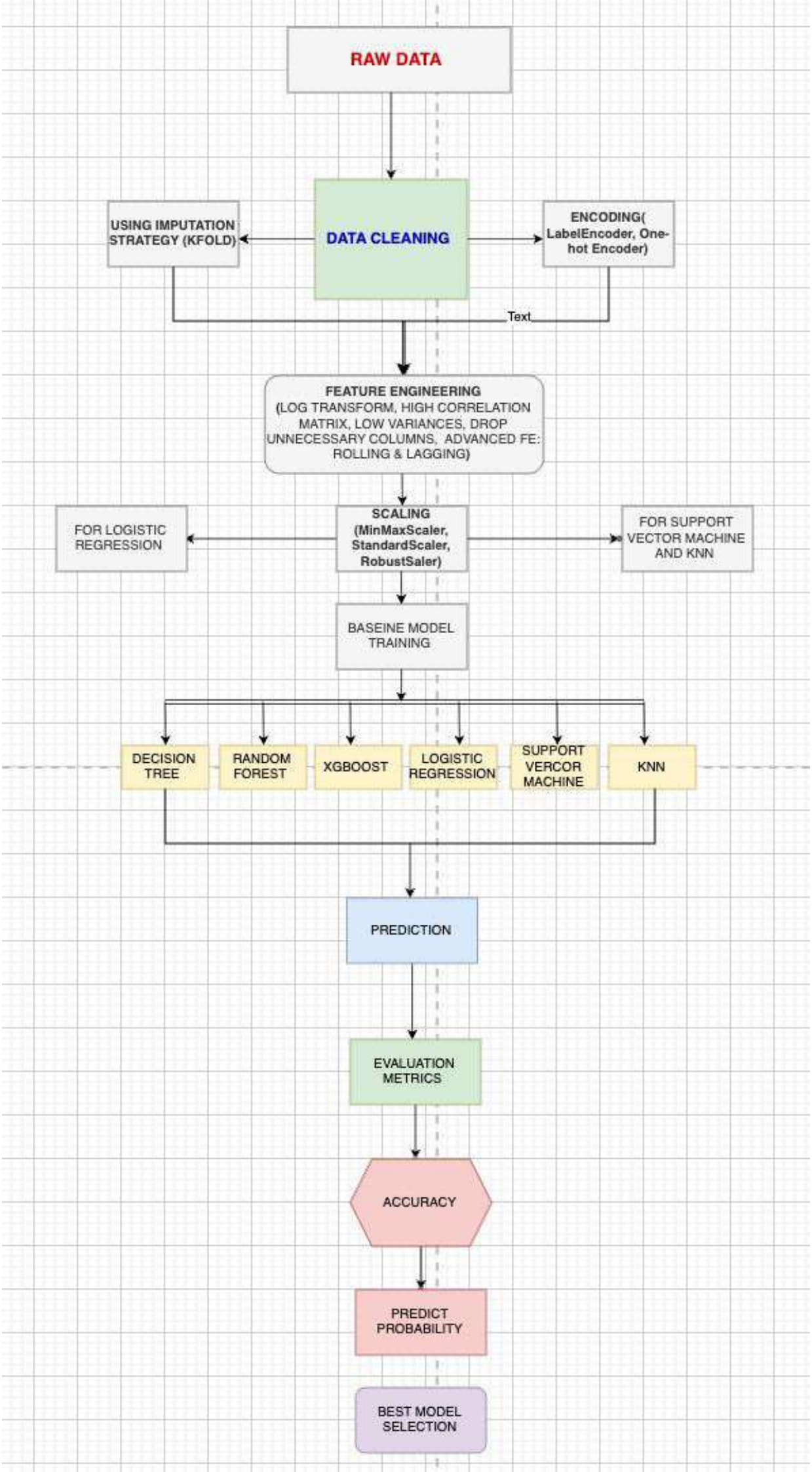
Natija:

age	age^2	churn
25	625	0
30	900	1

~~df.info()~~ df_poly.info() => Nechta ustun bor, qani null bor yoki yo'qligini, data type larni ko'rsatadi

Data leakage → Modelga kelajakdagi ma'lumotni berib qo'yish.


RAW DATA
DATA CLEANING
HANDING MISSING VALUES
ENCODING(LabelEncoder, One-hot Encoder)
TRAIN TEST SPLIT
SCALING (MinMaxScaler, StandardScaler, RobustSaler)
FOR LOGISTIC REGRESSION
FOR SUPPORT VECTOR MACHINE AND KNN
BASEINE MODEL TRAINING
DECISION TREE
RANDOM FOREST
XGBOOST
LOGISTIC REGRESSION
SUPPORT VERCOR MACHINE
KNN
PREDICTION
EVALUATION METRICS
ACCURACY, PRECISION, RECALL, F1_SCORE
BEST MODEL SELECTION


RAW DATA
DATA CLEANING
USING IMPUTATION STRATEGY (KFOLD)
ENCODING(LabelEncoder, One-hot Encoder)
Text
FEATURE ENGINEERING (LOG TRANSFORM, HIGH CORRELATION MATRIX, LOW VARIANCES, DROP UNNECESSARY COLUMNS, ADVANCED FE: ROLLING & LAGGING)
SCALING (MinMaxScaler, StandardScaler, RobustSaler)
FOR LOGISTIC REGRESSION
FOR SUPPORT VECTOR MACHINE AND KNN
BASEINE MODEL TRAINING
DECISION TREE
RANDOM FOREST
XGBOOST
LOGISTIC REGRESSION
SUPPORT VERCOR MACHINE
KNN
PREDICTION
EVALUATION METRICS
ACCURACY
PREDICT PROBABILITY
BEST MODEL SELECTION